A Comparative Study of Monolithic and Modular Architectures in a Dataset Driven Case Tracking Web Application

Mireya Francesca Vella
Institute of Information and Communication Technology
Malta College of Arts, Science and Technology
Paola, Malta
mireya.vella.i85114@mcast.edu.mt

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

A. Description of Theme and Topic Rationale

Backend architecture refers to how a server-side system is organised into components, responsibilities, and dependency boundaries. A common architectural choice in backend development is whether to build a monolithic system, in which most functionality is implemented within one tightly connected codebase, or a modular or layered system, in which responsibilities are separated into clearer units such as routing, services, and data access [5]. This choice is important because architecture affects how easily a system can be changed, tested, and maintained over time. Research on architectural smells, co-change behaviour, and modularity shows that unclear boundaries and poor structure can increase change propagation and reduce maintainability, which supports the need for evidence-based evaluation of architectural decisions [1], [3], [4].

This study focuses on a dataset-driven case-tracking application in order to compare these approaches in a controlled manner. Two backends are built with the same user-facing behaviour, including importing a synthetic JSON dataset into PostgreSQL, validating and normalising records, logging import runs, and exposing search and trend functionality. Comparing two implementations with identical requirements is appropriate because it allows differences in maintainability, testability, and operational performance to be measured under consistent conditions rather than assumed. This type of comparative evaluation is also consistent with prior work on software re-modularisation and architecture-level assessment [4], [7].

B. Positioning and Research Onion

This study adopts a pragmatist philosophy because it aims to develop and evaluate a working software artefact and to draw conclusions from practical, measurable evidence. This position is appropriate because the outcome is assessed using objective metrics collected from the implemented systems rather than the researcher's personal opinion. The study follows a deductive approach, as it seeks to test the predefined hypothesis that a modular or layered architecture provides better maintainability and testability than a monolithic design.

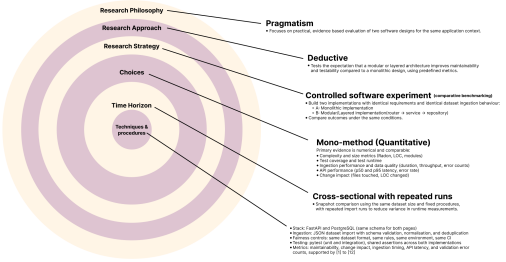

Fig. 1. Research Onion for the Study

The research strategy is a controlled software experiment based on comparative benchmarking. Two backend implementations are developed with identical requirements, identical dataset ingestion behaviour, and the same API contract. The analysis uses a mono-method quantitative strategy, since the research questions are addressed through numerical measures that can be compared across both architectures, including structural metrics, testing results, and performance timings. The time horizon is cross-sectional with repeated runs, meaning that both systems are evaluated at the same stage of completion while measurements are repeated to reduce random variation in runtime results. Recent research also shows growing interest in modular monolith design and stronger modular boundary practices, while emphasising the value of architecture evaluation through modularity metrics and re-modularisation analysis [4], [5], [7].

C. Background to this Research Theme

Architectural quality is closely related to maintainability, and architectural smells are recurring patterns that often indicate deeper design problems. Systematic evidence shows

increasing interest in detecting such smells as part of maintaining long-term system quality [1]. Studies further show that co-changes and modularity are useful for understanding how architecture affects system evolution and maintainability [3], [4].

D. Hypothesis, Research Aim, and Purpose Statement

This study tests the hypothesis that a modular or layered architecture is more maintainable and testable than a monolithic architecture.

The purpose of this controlled comparative experiment will be to test the theory that architectural modularity improves software quality by relating backend architecture type to maintainability and testability outcomes, while controlling for identical requirements, dataset, database schema, API contract, test suite, and runtime environment across both implementations. The independent variable will be backend architecture type, defined as a monolithic backend or a modular or layered backend. The dependent variables will be maintainability and testability, defined using tool-based structural measures and automated test evidence.

II. Literature Review

This section reviews the methodologies adopted in recent studies related to software architecture quality and modularity, with a focus on how researchers evaluate architectural smells, maintainability, modularity, and remodularisation. To provide a structured and comparative lens, this review follows the logic of the research onion by considering research philosophy, methodological approach, strategy, time horizon, and evaluation procedures used to validate architecture-related hypotheses [1], [5].

Across the reviewed works, a largely positivist stance is evident, prioritising objective and measurable outcomes derived from software artefacts such as source code, repository history, and static analysis outputs [1], [3], [6]. Most studies follow a deductive approach by testing expectations such as whether architectural smells are associated with change behaviour, whether modularity can be captured through stronger quantitative metrics, or whether restructuring can improve software quality [3], [4], [6], [7]. The dominant strategies are empirical mining studies and tool-supported analyses, complemented by evidence synthesis through mapping studies and workshop-based reviews [1], [5]. Time horizons are commonly cross-sectional (single snapshot of systems) or version-based (repeated measures across releases) depending on the study design [3].

All sources included in this review are academic publications, including peer-reviewed journal articles, conference papers, and workshop papers [1]– [7]. Non-academic sources such as vendor documentation, architecture blogs, and practitioner case posts can support implementation decisions, but they are not treated as primary evidence here because they often do not report controlled variables, reproducible methods, or peer-reviewed evaluation.

A. Methodologies of Similar Studies

[1] conducts a systematic mapping study on architectural smell detection. The methodology applies structured search and selection procedures, then categorises studies by detection approaches, data sources, tool support, and validation methods. This is strong for breadth and transparency, and it supports positioning by showing that detection practices are diverse and not fully standardised. However, mapping studies do not directly test outcomes such as improved maintainability, so they contribute classification and gap identification rather than causal evidence.

[3] explores the relation between co-changes and architectural smells through repository mining. Co-change behaviour is used as a proxy for coupling and change propagation, then analysed alongside smell presence. The strength is that it is grounded in real maintenance activity. The limitation is that commit granularity and workflow conventions can distort co-change patterns, which can threaten internal validity if not carefully controlled.

[4] proposes M-score as an empirically derived modularity metric. The methodology focuses on improving measurement quality, which is important because modularity is often used as a dependent variable in architecture evaluation. Stronger metrics support clearer comparison across systems. A limitation is that modularity is multi-dimensional, so M-score should be interpreted alongside complementary indicators such as complexity, test outcomes, and change impact.

[6] investigates correlations between architectural smells and static analysis warnings. Methodologically, this triangulates architecture-level detection with another measurement source (static analysis), which can improve construct validity by showing alignment between different quality signals. However, correlations must be interpreted cautiously because static warnings vary in severity and may not uniformly reflect architectural structure.

[7] presents a search-based remodularisation case study at Adyen. Case study methodology supports practical relevance by evaluating restructuring in a realistic industrial setting. This provides strong context and feasibility evidence, but generalisability is reduced because results can depend on the organisation, codebase constraints, and local engineering practices.

[2] proposes an effort estimation approach for clustering-based remodularisation in a conference companion venue. It supports framing restructuring cost and feasibility, but it is lighter empirical evidence compared to full-length journal studies.

Similarly, [5] discusses modular monolith as a trend in a workshop context, supporting conceptual positioning rather than providing strong controlled evaluation.

B. Comparisons of Studies

Across the reviewed studies, a consistent emphasis on measurable evidence is visible, but three methodological families emerge, with differences in how strongly they support causal conclusions. Evidence synthesis work such as [1]

TABLE I
COMPARATIVE OVERVIEW OF STUDIES

Study	*Focus*	*Strategy & Time Horizon*	*Evidence Type*
[1]	Smell detection research landscape	Systematic mapping, cross-sectional	Academic synthesis
[2]	Remodularisation effort estimation	Conference companion, cross-sectional	Early-stage academic evidence
[3]	Co-change and smells	Empirical study, version history mining	Co-change (evolution) metrics
[4]	Modularity measurement (M-score)	Empirical metric derivation and evaluation	Modularity metric validation
[5]	Modular monolith trend	Workshop review / conceptual discussion, cross-sectional	Academic positioning
[6]	Smells and static warnings	Empirical study, cross-sectional	Smell detection + static analysis outputs
[7]	Search-based remodularisation	Industrial case study, cross-sectional	Applied evaluation (industrial evidence)

TABLE II
RESEARCH DESIGN AND EVALUATION TOOLS

Study	*Research Design*	*Data Collection*	*Evaluation Tools / Measures*
[1]	Secondary research (mapping)	Study selection and classification	Taxonomies of methods, tool support, evaluation approaches
[2]	Short conference companion paper	Conceptual framing of effort estimation for remodularisation	Effort estimation framing for clustering-based remodularisation
[3]	Quantitative empirical	Version history mining	Co-change analysis examined in relation to architectural smells
[4]	Quantitative metric study	Software structural data	M-score computation, modularity scoring, and metric evaluation
[5]	Conceptual / workshop-based review	Selected academic and grey literature discussion	Definitions, trends, and architectural positioning
[6]	Quantitative empirical (triangulation)	Smell detection + static analysis warnings	Correlation between smell indicators and static analysis warnings
[7]	Industrial case study	System data from a real organisation	Search-based remodularisation outcomes; structural improvement evidence

provides structured overviews and highlights gaps, but does not directly test architecture outcomes in controlled conditions. Repository-based empirical studies such as [3] and [6] provide scalable, tool-supported measurement, yet often produce association evidence due to limited control of confounding variables. Metric-based and case-study research such as [4] and [7] provides detailed and practical evaluation, but can be difficult to generalise across systems.

These differences matter for the present research because they justify a controlled comparative benchmarking design. By holding requirements, dataset ingestion rules, schema, API behaviour, and test procedures constant, stronger internal validity can be achieved than in observational mining studies, while still producing practical evidence relevant to real backend development.

III. RESEARCH METHODOLOGIES

This study evaluates whether a modular or layered backend architecture improves maintainability and testability compared to a monolithic backend when both implementations deliver identical application behaviour under controlled conditions. The study is applied because it produces two working software artefacts and evaluates them using objective evidence collected through automated tests, tooling, and repeated benchmark runs. The methodological choices reflect common approaches in architecture quality research, which often rely on tool supported measurement and empirical evidence to reason about maintainability, modularity, and change impact [1], [3], [4], [6], [7].

Based on this aim, the following research questions were formed:

RQ1: To what extent does a modular or layered backend architecture improve maintainability compared to a monolithic backend when implementing the same requirements?
RQ2: To what extent does a modular or layered backend architecture improve testability compared to a monolithic backend under the same test suite and contract?
RQ3: How does architecture type affect operational performance for dataset ingestion and API responsiveness under the same environment and workload?
RQ4: How does architecture type affect change impact when implementing a controlled change request on the same codebase features?

A. Research Design

A pragmatist philosophy is adopted because the study aims to produce practical conclusions from measurable evidence obtained from implemented systems. A deductive approach is used, as the study tests the predefined hypothesis that a modular or layered backend will be more maintainable and more testable than a monolithic backend, consistent with research linking architecture structure, smells, modularity, and change coupling to maintainability outcomes [1], [3], [4], [6], [7]. The strategy is a controlled software experiment (comparative benchmarking) in which two implementations are evaluated under the same constraints to reduce confounding factors. A mono method quantitative design is used, since the research questions are answered through comparable numerical measures. The time horizon is cross sectional with repeated runs, meaning both systems are evaluated at the same stage of completion while measurements are repeated to reduce runtime variance.

B. Environment and Technologies Used

Both backend implementations use FastAPI (Python) with PostgreSQL. The system includes a dataset ingestion pipeline that validates, normalises, deduplicates, and inserts records,

while logging import run summaries. Fairness is maintained through the same database schema and migrations, the same dataset format and rules, and identical API behaviour validated through shared integration tests. Evaluation evidence includes tool based structural measurement (including modularity indicators such as M score where applicable) [4], architecture quality indicators informed by smell research [1], [3], [6], and repeatable performance measurement for ingestion and API responsiveness.

C. Experiment Design

Two artefacts are implemented: (1) a monolithic backend in which routing, business rules, and data access are tightly integrated, and (2) a modular backend in which routing, business rules, and data access are separated into distinct modules with defined interfaces. Both versions are assessed under the same controls: identical requirements and outputs, identical database schema and migrations, identical dataset ingestion rules, and identical automated test procedures. A controlled change request is applied to both versions to measure change impact using files touched and lines changed, grounded in evolution and co change rationale [3], [7].

D. Validity, Reliability, and Generalisability

Internal validity is strengthened through fairness controls, reducing the likelihood that differences arise from feature drift or environment variation, which is a common limitation in observational architecture research [3], [6]. Construct validity is supported by triangulating multiple indicators for maintainability and testability rather than relying on a single metric, consistent with concerns in modularity measurement [4]. Reliability is supported through automation and repeated benchmark runs. Generalisability is limited to the chosen application and stack, but transferability is supported by clearly defined procedures and reporting, consistent with how industrial and case based evidence is typically contextualised [7].

E. Ethical Considerations

This study does not involve human participants. The dataset is synthetic, reducing the risk of handling personal or sensitive data. The study reports methods, controls, and limitations transparently, and acknowledges tool limitations to avoid overstating conclusions [1], [4].

References

[1] H. Mumtaz, P. Singh, and K. Blincoe, "A Systematic Mapping Study on Architectural Smells Detection," Journal of Systems and Software, vol. 173, Art. no. 110885, 2021, doi: 10.1016/j.jss.2020.110885.

[2] A. J. J. Tan, C. Y. Chong, and A. Aleti, "Closing the Loop for Software Remodularisation: REARRANGE, An Effort Estimation Approach for Software Clustering Based Remodularisation," in Companion Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE-Companion), 2023, pp. 326–327, doi: 10.1109/ICSE-Companion58688.2023.00090.

[3] D. Sas, P. Avgeriou, R. Kruizinga, and R. Scheedler, "Exploring the Relation Between Co-changes and Architectural Smells," SN Computer Science, 2021, doi: 10.1007/s42979-020-00407-5.

[4] E. Pisch, Y. Cai, R. Kazman, J. Lefever, and H. Fang, "M-score: An Empirically Derived Software Modularity Metric," in Proc. ACM/IEEE Int. Symp. on Empirical Software Engineering and Measurement (ESEM), pp. 382–392, 2024, doi: 10.1145/3674805.3686697.

[5] R. Su and X. Li, "Modular Monolith: Is This the Trend in Software Architecture?," in Proc. 1st Int. Workshop on New Trends in Software Architecture (SATrends '24), 2024, doi: 10.1145/3643657.3643911.

[6] M. Esposito, M. Robredo, F. Arcelli Fontana, and V. Lenarduzzi, "On the Correlation Between Architectural Smells and Static Analysis Warnings," Software Quality Journal, 2025, doi: 10.1007/s11219-025-09730-7.

[7] C. Schröder, A. van der Feltz, A. Panichella, and M. Aniche, "Search Based Software Re Modularization: A Case Study at Adyen," in Proc. IEEE/ACM 43rd Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2021, pp. 81–90, doi: 10.1109/ICSE-SEIP52600.2021.00017.